CS 4300/5300 Software Engineering

Common Sprint Requirements

Each sprint is two weeks long. Make sure that you work through both weeks of the sprint to achieve your tasks.

1 Sprint Planning

1. At the beginning of the sprint, meet with your group to do **backlog grooming** and **prioritization**. Mark stories that are MVPs (minimum viable product).
2. Begin the Sprint by looking over your backlog. Add stories or divide stories as needed—some of this may include playing Planning Poker in your team to determine points and need for division. Record the changes in GitHub.
3. Meet with your customer to identify the stories they want you to implement. Prioritize and confirm with them (if you can).
4. The number of stories that you will complete each sprint depends on the difficulty of the stories. Select as many stories as you hope to get done—if you don't meet your expectations, that's okay, but each person should have a story/stories to work on. You will balance the workload between each of the sprints. At least two larger stories should be implemented and fully tested each sprint. Pairs of students may work on features that are especially large.
5. Move the selected tasks from Backlog to the current Sprint.

Note: Keep the FIRST and SMART principles in mind—it is fine to divide stories and tasks so that they are achievable and measurable. MVPs should be prioritized. We only have 4 iterations, so pick what's really important! It is good to have a few extra MVPs/non-MVPs assigned to the sprint that can be ready to be assigned and moved on as initial tasks are completed.

2 Development Requirements

1. Every story in the sprint has one main owner and comes with automated tests. Use the kinds of tests that fit the feature: unit, integration, BDD, or a mix.
2. Follow **Red-Green-Refactor** for new behavior: write a test and watch it fail, write just enough code to pass, then clean up while the tests stay green. Bigger refactoring can wait until later in the semester.
3. Your **team** is responsible for **full stack development**: the front end, the back end, and the right automated tests across the application. You don't have to work on every layer yourself, but whatever you work on should be real, tested, and integrated.
4. Communication and the use of branches will be key. Push, pull, and merge to main frequently. Protections have been placed on the main branch, so approval must be given for your merge to main to go through. **Work early** so that your branch is submitted in time for approval.
5. As you finish tasks, move them along to completion in your project management tool.

3 Demo Expectations

For each sprint demo, you must:

1. Share the URL to your production deployment so we can follow along
2. Tell us about your application and why you're making it
3. Go through GitHub Projects and show us what work you completed based on user story
4. Demo the work you completed on the production environment showing the feature in production and showing us the test cases used to test each requirement
5. Get customer feedback

4 Submission Requirements

On Canvas (1 person)

- Submit your GitHub repository address
- Submit your application's production URL
- Submit a short report of what was implemented and tested. You may have not completed everything—include that in your report discussing what you will do.
- If your plan changed since your last submission, explain why
- If you are having issues with some of your tests, document why

On GitHub

- Make sure that all project files including your tests and code are committed and pushed to GitHub on the main branch
- Make sure your work shows up in GitHub under your own name (learn more). If you pair, add a `Co-authored-by:` line so both people get credit. We grade what you contributed, not how many times you pushed.
- **Tag your git repository revision** with "sprint$i$" where i is the sprint number
  - To tag the current revision: `git tag -a sprint1 -m "Sprint 1"`
  - **Warning:** You must explicitly push your tags with the command `git push --tags`. Before you submit, please make sure that we can checkout the tag—you can try this from the command line and also check the GitHub website to double check that the tag is there.

On GitHub Projects

Ensure project management tools are updated, cards are moved to done, and acceptance criteria has been developed for each card as gherkin scenarios. Be prepared to demo these scenarios during class.

On Slack

One team member must post to your Dev_Cust Group on Slack with information about what was implemented and your application's URL. Customers will see and can write posts to/from their developers.

AI Documentation

You must document in your project README where and when you got help from AI: the tool you used, what it helped with, and how you used the result. You do not need to include transcripts or chat logs.

Peer Evaluations

Complete your peer evaluations! The activity opens immediately after the sprint deadline and has multiple stages to complete. See Section 5 for how they count.

5 How Your Sprint Is Graded

Your sprint grade has three parts: finishing your Kritik peer evaluations, your team's result, and your own engineering work.

The formula

First work out two percentages:

- **Team %** = your team's rubric points ÷ 88
- **Individual %** = your individual points ÷ 30

Then:

Sprint grade = Kritik (12) + 88 × (**65%** × Team % + **35%** × Individual %)

- **Kritik (12 pts):** you get these for finishing every stage of the Kritik peer evaluation. They're the same 12 points listed in each sprint's rubric.
- **Team score (88 pts):** everything else in that sprint's rubric (tests, coverage, CI, deployment, and so on). Everyone on the team gets the same team score.
- **Individual score (30 pts):** the engineering work *you* did this sprint. See below.

Example. Your team earns 80 of 88 (90.9%), you earn 24 of 30 (80%), and you finish Kritik (12/12).
Sprint grade = $12 + 88 \times (0.65 \times 0.909 + 0.35 \times 0.80) = 12 + 76.6 =$ **88.6**.

Below 15/30 individually. If your individual score is under 15, your instructor double-checks it before grades go out. That means looking again at your GitHub work, your teammates' Kritik comments, and anything you've told us about your situation. Once the score is confirmed, the normal formula is used.

Your individual score (30 points)

Category	Pts	What earns the points
Engineering substance	10	How hard, deep, and important your work was. Real front end, back end, database, security, deployment, CI/CD, AI integration, and test infrastructure work can all earn full credit. You don't have to touch every layer. A button, header, or styling change on its own scores low.
Delivered impact	8	Your work is merged to `main`, working, and in the sprint release by the sprint tag. Unmerged work gets credit for progress, not for delivery. If your work was ready but blocked by something outside your control, tell us and we'll look at it.
Testing & verification	7	You tested your own work with the kinds of tests that fit it: unit, integration, or BDD. Testing edge cases, errors, and permissions, with CI passing, earns the top of the range. You don't need every kind of test for every feature.
Collaboration & review	5	Reviewing teammates' pull requests, helping debug, fixing merge conflicts, and helping get other people's work integrated.
Total	**30**	

How points are given in each category. Each category is scored on the quality and completeness of your contribution, based on the evidence we have:

Category	Full	Most	Some	Little or none
Engineering substance (10)	9–10	6–8	3–5	0–2
Delivered impact (8)	7–8	5–6	2–4	0–1
Testing & verification (7)	6–7	4–5	2–3	0–1
Collaboration & review (5)	4–5	3	1–2	0

You need both code and tests for full credit. Code alone, or tests alone, can't earn 30/30.

We don't count activity. Commits, lines of code, number of pull requests, and story points help us understand your work, but they're never turned into points. One solid, well-tested feature beats fifty tiny commits.

What we look at

A working feature isn't enough on its own. We follow your work from start to finish:

Plan → Own → Build → Test → Review → Merge → Deploy

That trail comes from GitHub: the cards assigned to you on your Project board, your pull requests, your code and tests, CI results, code reviews, and what made it to production. If you did the work but none of it shows up there, we can't give you full credit for it.

Make your work visible

- **Assign yourself** to the cards you own on the Project board, and keep their status up to date.
- **Link every pull request to its card or issue** (for example, write "Closes #12" in the PR description).
- **Put your tests in the same pull request** as the feature they test.
- **Push to GitHub under your own name.** If you pair program, add a `Co-authored-by:` line to the commit so your partner gets credit too.
- **Review your teammates' pull requests** and leave real comments.
- **Work across the whole sprint,** not in one push the night before.

You don't need to make extra commits or pad your activity. Just work the way described above, and your contribution will be easy to see.

Kritik peer evaluations

Kritik is used in two ways:

1. **Finishing it earns the 12 points.** Whether your reviews are kind or critical doesn't change your score. Just be fair and specific.
2. **Your reviews tell us things GitHub can't**, like pairing, debugging, helping a teammate get unstuck, or integration work. Answer this question: *"Is there anything about this sprint that the GitHub history alone would give the wrong impression about?"*

You are not grading your teammates. Good context can raise someone's individual score when GitHub doesn't show everything they did. Negative comments never lower a score by themselves; they just make us look more closely at the GitHub evidence. What teammates write about you stays confidential.

Evidence confidence

Your feedback also rates our **evidence confidence** as **High** (85% or more), **Medium** (65–84%), or **Low** (under 65%). It says how easy it was to trace your work in GitHub. It goes down when commits are hard to attribute, cards aren't linked to pull requests, or pairing isn't recorded. It is *not* a measure of how good your work was, and it doesn't change your grade. Low confidence just means we take a closer look by hand.

Life happens

If illness, a family emergency, or another serious situation affects your sprint, contact your instructor as soon as you can. Depending on what happened, we may excuse your individual score, extend your deadline, or give you equivalent engineering work instead. If your individual score is excused, your grade is your Kritik points + 88 × Team %. Your team's expected scope is adjusted to match. Personal details are never written into your feedback.

Your feedback

After each sprint, you'll get a private written report explaining your individual score, what was strong, and what would raise it. Your official grade is in Canvas.